BREAKING BAD CTF: HEISENBERG'S LAB - WALKTHROUGH

📋 Game Overview

- **Total Levels:** 8
- **Difficulty:** Easy → Hard
- **Platform:** Android (Kotlin + Compose)
- **Flag Format:** `BreakingBadCTF{...}`
- **Required Tools:** `adb`, `jadx`, `Python 3`, `Frida`, `Burp Suite`

Essential Tools

- **`adb`**: Interacting with the device, setting properties, and pulling files.
- **`jadx-gui`**: Decompiling the APK to read source code and resources.
- **`Frida`**: Injecting scripts into the running app to bypass security.
- **`CyberChef`**: Quick decoding of Base64, AES, and Vigenère ciphers.

LEVELS :

- **Level 1:** The RV Key
- **Level 2:** Los Pollos Menu
- **Level 3:** Better Call Saul
- **Level 4:** The Blue Sky
- **Level 5:** Gus's CCTV
- **Level 6:** Walt's Phone
- **Level 7:** Hector's Bell
- **Level 8:** The Empire {The Heisenberg Legacy}

Quick Reference: All Flags

#	Level Name	Technique	Flag
1	**The RV Key**	Strings + Base64	BreakingBadCTF{rv_key_168247}
2	**Los Pollos Menu**	APK Assets + Caesar + B64	BreakingBadCTF{pollo_fried_9281}
3	**Better Call Saul**	adb logcat + AES-CBC	BreakingBadCTF{saul_1-800-255-5299}
4	**The Blue Sky**	System Properties + XOR	BreakingBadCTF{blue_63%_pure}
5	**Gus's CCTV**	Exported Activity + LSB Stego	BreakingBadCTF{cctv_bypass_gus}
6	**Walt's Phone**	OSINT + PNG EOF Stego	BreakingBadCTF{heisenberg_737}

7	**Hector's Bell**	Frida Hooking + Anti-debug	BreakingBadCTF{ding_000001}
8	**The Empire**	SQLi + Vigenère + Frida	BreakingBadCTF{saul_B3MBP3N_F4LW_saul_1962}

Level-by-Level Walkthrough

Level 1: The RV Key (Easy)

Concept: Static resource analysis and multi-layer Base64 encoding.

- **The Challenge:** The registration key is hidden in strings.xml, protected by an MCQ gate and encoding.
- **Solution:**
  1. Decompile the APK using jadx-gui.
  2. Locate rv_key_encoded in res/values/strings.xml.
  3. **Decoding Pipeline:** Base64 decode → Reverse string → Base64 decode.
- **Flag:** BreakingBadCTF{rv_key_168247}

Level 2: Los Pollos Menu (Easy)

Concept: Asset extraction and hybrid encryption.

- **The Challenge:** Secrets are hidden in assets/ in a file with a Base64-encoded name.
- **Solution:**
  1. Extract the APK and find ZmxhZ3BhcnQx.txt (which decodes to "flagpart1").
  2. Decode the file contents to get flag{wvssv_ZnJpZWQ=_9281}.
  3. Apply a **Caesar shift of -7** to "wvssv" (pollo) and Base64 decode "ZnJpZWQ=" (fried).
- **Flag:** BreakingBadCTF{pollo_fried_9281}

Level 3: Better Call Saul (Medium)

Concept: Analyzing runtime logs and AES decryption.

- **The Challenge:** The app emits encrypted logs under the tag SaulBurnerService.
- **Solution:**
  1. Run adb logcat | grep SaulBurnerService while the level loads.
  2. Find the key api_client_identity (BETTER_CALL_SAUL) in strings.xml.

    3. Decrypt the log's ciphertext using **AES-CBC** mode.
- **Flag:** BreakingBadCTF{saul_1-800-255-5299}

Level 4: The Blue Sky (Medium)

Concept: Manipulating Android system properties.

- **The Challenge:** The "reactor" only activates if a specific system property is set.
- **Solution:**
    1. Set the precursor via terminal: adb shell setprop debug.lab.precursor methylamine.
    2. The app reveals a payload: W11XW0pXU0Y1Wld4bGNpY3lRems5Y0hWeVp3PT0=.
    3. **Decoding:** Base64 decode → Reverse → Base64 decode → XOR with "methylamine".
- **Flag:** BreakingBadCTF{blue_63%_pure}

Level 5: Gus's CCTV (Medium)

Concept: Exploiting exported activities and LSB steganography.

- **The Challenge:** A hidden surveillance screen is accessible via a manifest vulnerability.
- **Solution:**
    1. Identify CctvActivity as exported="true" in the AndroidManifest.xml.
    2. Force-start it: adb shell am start -n com.ctf.breakingbad/.CctvActivity.
    3. Extract assets/cctv_feed.png and use a Python script to pull the **Least Significant Bit (LSB)** from the red channel.
- **Flag:** BreakingBadCTF{cctv_bypass_gus}

Level 6: Walt's Phone (Medium)

Concept: Multi-step forensics and OSINT.

- **The Challenge:** A series of physical interactions and file discoveries.
- **Solution:**
    1. **Secret Knock:** Rapidly tap the periodic table 6 times.
    2. **OSINT:** Zoom out in the surveillance view to find the "ALBUQUERQUE" billboard (XOR key).
    3. **Forensics:** Pull waltscrete.txt, XOR decrypt it to find a hint about Hank_Schrader.png, then read data appended after the IEND chunk of the image.
- **Flag:** BreakingBadCTF{heisenberg_737}

Level 7: Hector's Bell (Hard)

Concept: Dynamic instrumentation with Frida.

- **The Challenge:** The level is protected by anti-debugging checks that kill the app if Frida is detected.
- **Solution:**
    1. Start `frida-server` on the device.
    2. Identify the obfuscated `checkAccess()` method in `Level7ScreenKt`.
    3. Use Frida to hook the method and force it to return `true`.

Frida Hook Script (hook_bell.js)

```
Java.perform(function() {
  var cls = Java.use(
    "com.ctf.breakingbad.ui.screens.Level7ScreenKt"
  );
  cls["Level7Screen$checkAccess"].implementation = function() {
    console.log("[*] checkAccess hooked → returning true");
    return true;
  };
  console.log("[*] Hook applied. Press RING THE BELL.");
});
```

- **Flag:** `BreakingBadCTF{ding_000001}`

Level 8: The Empire (Hard)

Concept: A final 4-part challenge combining all previous skills.

- **The Fragments:**
    1. **SQLi:** Bypass login with `' OR 1=1 --` and hex decode the "secret ingredient" → **saul**.

1	Open Sub-Level 1 — Madrigal Electromotive login portal
2	Enter in username field: admin' OR 1=1 --
3	Logged in — SQL result shows users table
4	Execute: SELECT * FROM secret_ingredients
5	Find row: distributor_id | hex_signature: 0x53 0x61 0x75 0x6c
6	Decode hex: 0x53=S 0x61=a 0x75=u 0x6c=l → Saul
7	Type "Saul" in the input field → Fragment 1 saved

    2. **OSINT:** Identify Gray Matter's founding year → **1962**.

1	Open Sub-Level 2 — Mike's secure inbox
2	Tap the suspicious email from "Gray Matter Technologies"
3	Read the email — it references Vamonos Pest and Gray Matter founding year

3. **Vigenère:** Decrypt the ransomware payment ID using the key "SALAMANCA" → ***B3MBP3N_F4LW***.

Vigenère Python Solver

```
def vig_decrypt(ct, key):
    k, out, ki = key.upper(), [], 0
    for c in ct:
        if c.isalpha():
            b = ord("A") if c.isupper() else ord("a")
            s = ord(k[ki % len(k)]) - ord("A")
            out.append(chr((ord(c)-b-s+26)%26+b)); ki+=1
        else: out.append(c)
    return "".join(out)
print(vig_decrypt("_T3MMP3Z_F4YY_", "SALAMANCA"))
# Output: _B3MBP3N_F4LW_
```

4. **Frida:** Hook the Cartel API to capture a Base64-encoded logcat fragment → **saul_**.

Frida Hook for Sub-Level 4 (hook_cartel.js)

```
Java.perform(function() {
  var cls = Java.use("com.ctf.breakingbad.SubLevel4ActivityKt");
  cls["SubLevel4Screen$checkSecurityStatus"].implementation = function() {
    console.log("[*] checkSecurityStatus → true");
    return true;
  };
  console.log("[*] Hook applied. Press RUN HEALTH CHECK.");
});
```

- **Assembly:** Combine fragments in order: `saul` + `_B3MBP3N_F4LW_` + `saul_` + `1962`.
- **Flag:** `BreakingBadCTF{saul_B3MBP3N_F4LW_saul_1962}`